Cross-Browser Orchestration of High-Performance Computing Workloads via LLM Interface Interception

Executive Summary

The orchestration of high-performance computing (HPC) workloads traditionally requires highly specialized knowledge of secure shell (SSH) protocols, workload managers such as Slurm, and complex containerization runtimes like Apptainer. Conversely, modern large language model (LLM) interfaces—most notably the official Anthropic Claude web application—offer highly optimized, multimodal user experiences but remain fundamentally disconnected from localized, secure academic data enclaves. Bridging this gap by seamlessly intercepting prompts from a production LLM web interface and redirecting them to a generalized university HPC grid represents a paradigm shift in computational research environments.

This comprehensive analysis confirms the absolute technical feasibility of a specialized architecture that links a commercial LLM frontend directly to a secure academic backend, utilizing the infrastructure of Wayne State University (https://ondemand.grid.wayne.edu/) as a primary deployment blueprint[1]. By leveraging a Manifest V3 cross-browser extension, Open OnDemand (OOD) reverse proxies, and a generalized containerized backend running FastAPI, it is possible to hijack the native web interface of consumer LLMs[1]. This architecture allows researchers to leverage the pristine, production-grade user interface of the claude.ai web application while redirecting the actual prompt payloads to an isolated HPC compute node[1].

Crucially, this system relies on several architectural mechanisms that function as deployment "cheat codes"—most notably the innate capacity of modern web browsers to share authentication tokens via HttpOnly cookies across active tabs, and the relative routing capabilities of the Open OnDemand gateway[1]. Validating the premise of shared authentication reveals that when a user is actively authenticated to their university grid in one tab, the browser extension can seamlessly tunnel high-bandwidth, multimodal data from the claude.ai tab directly into the HPC cluster without requiring the user to manage SSH keys, OAuth flows, or localized configuration files[1]. By executing a targeted sequence of Document Object Model (DOM) interception, asynchronous message passing, deterministic schema validation, and unidirectional event streaming, this framework seamlessly projects the analytical capabilities of a multimodal LLM into a highly secure, localized execution environment[1].

The Browser-Agnostic Extension Framework

To achieve universal deployment across Google Chrome, Apple Safari, Mozilla Firefox, and Microsoft Edge, the architecture requires a unified compilation pipeline capable of handling differing security paradigms and namespace conventions[1]. The fragmentation of the browser

extension ecosystem, exacerbated by the transition from Manifest V2 to Manifest V3, necessitates a robust build engine.

Evaluating Cross-Browser Build Engines

The web extension development landscape features several modern frameworks designed to abstract the complexities of cross-browser compatibility. When evaluating the specific requirements of a robust, widely deployable academic tool, the Web Extension Toolkit (WXT) framework serves as the optimal foundation[1]. Built on top of the Vite build engine, WXT allows developers to maintain a single TypeScript-based codebase that dynamically compiles into the appropriate manifest package depending on the target browser[1]. WXT streamlines development through file-based routing and auto-imports, eliminating the boilerplate associated with manual manifest generation[1].

Alternative frameworks present limitations in a truly browser-agnostic context. Plasmo offers exceptional React integration and is widely used for Chrome Manifest V3 extensions, but its official support documentation lists Firefox Manifest V3 as experimental and requires manual wrappers for Safari targets[1]. Extension.js provides a zero-configuration approach but relies on compiling whatever is explicitly declared in a pre-existing manifest rather than generating it dynamically from file structures[1].

Framework	Build Engine	Cross-Browser Output Capability	Manifest Generation Strategy	Primary Architecture
WXT	Vite	First-class Chrome, Firefox, Safari, Edge	Automatic via Config/Files	File-based Entrypoints
Plasmo	Parcel-based (Custom)	Chrome/Edge (Firefox/Safari require manual shims)	Automatic via Config	File-based Entrypoints
Extension.js	Webpack/Vite	Chrome, Firefox, Edge	Direct Read/Compile	Config-based
CRXJS	Vite	Chrome/Edge (Brittle on upgrades)	Direct Read/Compile	Config-based

Bridging Namespace Disparities via Polyfills

The actual JavaScript execution environment differs significantly across browser vendors. Firefox utilizes a browser namespace with Promise-based APIs by default[1]. Conversely, Chrome has historically utilized callback-based APIs under the chrome namespace, although modern versions are incrementally supporting Promises[1]. Edge largely follows Chrome's conventions due to its underlying Chromium engine[1].

Attempting to manage these disparities through conditional logic results in brittle codebases. WXT inherently simplifies this by providing unified wrappers for different browser storage and messaging APIs[1]. The architecture heavily relies on the mozilla/webextension-polyfill library, which augments the WebExtensions API methods, ensuring that all cross-browser interactions operate under a unified, promise-based namespace[1]. Furthermore, because the target web application (claude.ai) operates as a Single Page Application (SPA) using React, traditional page load listeners are insufficient for triggering script injections[10]. The WXT framework natively handles SPA routing via the wxt:locationchange event listener, ensuring that the DOM interception logic is reliably re-initialized as the user navigates between different chat sessions without triggering full page reloads[10].

Manifest V3 Execution Worlds and DOM Interception

The core mechanism of this orchestration system relies on capturing the user's prompt before it is transmitted to Anthropic's cloud servers. Modern web applications utilize the window.fetch API or XMLHttpRequest for outbound network requests. Intercepting these requests requires injecting a JavaScript payload into the execution context of the target web application. This presents a unique challenge under the strict architectural constraints of Manifest V3.

Navigating the Isolated and Main Worlds

In Manifest V3, content scripts run in an ISOLATED execution world by default[1]. This architectural decision dictates that while the content script shares the DOM with the host page, it maintains a completely separate JavaScript execution environment and global scope[1]. Consequently, monkey-patching window.fetch within an ISOLATED world will only intercept fetch calls made by the extension itself; it cannot observe or intercept the fetch calls executed by the primary web application running in the MAIN world[1].

To overcome this fundamental limitation, the extension must explicitly declare a content script destined for the MAIN execution world. Modern Manifest V3 specifications allow extensions to inject scripts directly into the MAIN world using the world: "MAIN" configuration directive in the manifest.json file, or programmatically via the chrome.scripting.executeScript method[1]. The WXT framework facilitates this through its injectScript utility, which dynamically creates a script element on the page pointing to a bundled JavaScript file, forcing the browser to load the logic directly into the host page's main execution context[10].

However, injecting scripts into the MAIN world exposes the extension to the host page's Content Security Policy (CSP). If a page enforces strict Trusted Types or blocks dynamic compilation via unsafe-eval restrictions, traditional script injection techniques fail[15]. By relying on statically bundled scripts registered in the manifest rather than dynamically evaluating code strings, the architecture securely navigates these CSP restrictions while achieving synchronous execution

before the page's native scripts can lock down the networking APIs[15].
Once successfully injected into the MAIN world, the script executes a precise sequence of operations to hijack the data pipeline. Initially, it stores a reference to the original, native window.fetch function to ensure that unrelated network traffic can be processed normally. It then overwrites the global window.fetch variable with a custom asynchronous proxy function[1]. Every outbound network request is subsequently funneled through this proxy. The proxy function inspects the request URL; if the URL matches the signature of a conversational submission endpoint—such as routes containing /api/organizations/ or /chat_conversations/—the interception logic is triggered[1].

Multimodal Payload Extraction and Visual Processing

When the interception logic triggers, the proxy function parses the intercepted Request body, extracting the text prompt and any associated multimodal data[1]. The Claude API natively supports robust multimodal reasoning, allowing users to upload documents, spreadsheets, and images directly into the chat interface[18].

Document and Image Serialization

Understanding how the frontend serializes this data is critical for the interception pipeline. Claude distinguishes between text extraction and true multimodal reasoning[18]. Text extraction applies to standard formats such as TXT, DOCX, RTF, and CSV, where the frontend parses the text line-by-line and embeds it as standard string content within the payload[18]. Conversely, multimodal reasoning activates when processing PDFs or image files (JPEG, PNG, GIF, WebP)[18]. In this mode, the model interprets both text and visuals simultaneously, detecting embedded graphs and layout structures[18].
During a standard chat session, image files and PDFs are processed entirely by the local browser frontend, converted to Base64-encoded strings, and embedded directly within the JSON payload sent via the fetch API[19]. The injected interceptor script extracts this Base64 data from the content_block arrays within the payload[19]. This architectural reality is highly advantageous: the browser extension does not need to perform complex file parsing, manage cross-origin file uploads, or coordinate separate data transfers to the HPC cluster. The claude.ai frontend has already flawlessly packaged the multimodal data into a localized, serialized string that can be easily transported over an HTTP message bus[7].
The system must account for the specific tokenization constraints of Claude's vision models. Claude views images in patches rather than individual pixels, where each patch constitutes a 28x28-pixel block representing a single visual token[19]. The Base64 payloads intercepted by the extension can be substantial, with single images reaching up to 10 Megabytes in size[19]. The extraction logic must efficiently buffer these large strings without causing memory leaks in the browser tab[21].
Furthermore, advanced capabilities such as the Web Fetch Tool allow Claude to retrieve full content from specified web URLs to augment its context[22]. If the user prompt utilizes this tool, the intercepted payload will contain the tool invocation schema[22]. The backend HPC script must be prepared to honor these tool calls, executing the necessary HTTP fetches natively from the

compute node to simulate the behavior the user expects from the official web application[22].
Once the prompt and multimodal payloads are extracted, the outbound request to the external commercial cloud server is actively aborted[1]. Crucially, to prevent the React-based front-end user interface from crashing or displaying a fatal network timeout error, the interceptor immediately constructs and returns a mock Response object[1]. This mocked response simulates a successful API transaction, returning a generic status message indicating that the prompt is being routed to the generalized HPC grid worker, thereby maintaining UI stability while the actual computation is redirected[1].

Secure Cross-Origin Message Transport Architecture

Once the payload is successfully extracted within the MAIN world, it must be transmitted to the HPC grid. Because of strict Cross-Origin Resource Sharing (CORS) and Content Security Policy (CSP) restrictions enforced by production web applications, the injected MAIN world script cannot communicate directly with the HPC endpoint[1].
To circumvent this securely, the architecture utilizes a sophisticated, multi-hop asynchronous message bus. While early iterations of extension architecture relied on window.postMessage() to broadcast payloads across the DOM, this method presents significant security vulnerabilities[1]. If a generic postMessage is utilized, page scripts running natively on the host domain can potentially sniff the payload, exposing the researcher's proprietary data[15].
Instead, the system relies on a cryptographic event bridge. The MAIN world script dispatches a CustomEvent across the DOM[15]. To prevent interception by native page scripts, the event type is dynamically generated using a UUID or secure cryptographic secret established during the initial script injection[15]. This ensures that only scripts possessing the exact secret key can listen for the event[15].
An ISOLATED world content script, operating concurrently, listens for this specific, secret-keyed CustomEvent[15]. Because it operates in the isolated execution environment, it possesses elevated privileges allowing it to access the browser's extension APIs[24]. The ISOLATED script receives the multimodal payload and leverages browser.runtime.sendMessage() to relay the data out of the DOM entirely and into the extension's Background Service Worker[1]. The Background Service Worker, operating continuously as an invisible background process with elevated network privileges defined in the manifest.json, then assumes responsibility for initiating the HTTP POST request to the HPC grid[1].

Zero-Credential Authentication and Open OnDemand Routing

The most significant bottleneck in deploying remote execution pipelines lies in user authentication. Traditional paradigms require researchers to generate SSH key pairs, manage localized ~/.ssh/config files, or generate long-lived API tokens[1]. This introduces massive user friction and exposes significant security risks if keys are mishandled or leaked[1].
The defining "cheat code" of this architecture is its absolute reliance on the shared cookie state maintained by the web browser[1]. The initial setup assumes the user has the claude.ai interface

open in one tab, and their university's Open OnDemand portal (e.g., https://ondemand.grid.wayne.edu/) authenticated in a separate tab[1].

Validating the Shared Authentication Mechanism

When a researcher logs into the OOD portal via their institution's Single Sign-On (SSO), OpenID Connect (OIDC), or Shibboleth provider, the Apache backend generates secure, HttpOnly session cookies (such as mod_auth_openidc_session) to track the authenticated state[1]. The fundamental mechanics of web browsers dictate that cookies are scoped to specific domains and paths, not to individual browser tabs.

The extension's Background Service Worker operates independently of the individual tabs but retains access to the browser's unified cookie jar for domains where it has been granted explicit host_permissions[17]. When the Service Worker constructs the outbound fetch() request containing the intercepted LLM payload, it explicitly includes the credentials: 'include' directive[1]. This directive forces the browser to automatically attach the active Wayne State OOD session cookies to the outbound payload[1].

Upon arrival at the OOD gateway, the built-in Apache reverse proxy reads the cookie, validates the cryptographic signature against the OIDC provider, confirms the user is authorized to access the cluster, and forwards the traffic into the internal network[1]. At no point does the browser extension, the LLM, or the backend execution engine ever read, copy, prompt for, or store a password, OAuth token, or SSH key[1]. This zero-credential routing represents a massive reduction in user friction, relying entirely on the pristine, battle-tested session management of the underlying browser architecture.

The Reverse Proxy Routing Mechanisms

Once authenticated at the perimeter, the OOD gateway must route the HTTP traffic through the university firewall directly to an isolated, active compute node running the user's allocated job[1]. OOD facilitates this via its mod_ood_proxy Apache module, which relies on complex Lua scripts (proxy.lua) to dynamically tunnel HTTP and WebSocket traffic to internal IP addresses[1].

The browser extension must dynamically construct the target URL utilizing one of two primary OOD proxy configurations: absolute proxying (node) or relative proxying (rnode).

- **Absolute Proxying (node URI):** A request structured as /node/<host>/<port>/<path> reverse proxies the traffic to the backend server utilizing the full, untouched URL path[1]. For instance, a request sent to /node/compute01/8080/api/run is delivered to the backend server exactly as /node/compute01/8080/api/run. This requires the backend Python application to be explicitly aware of the sub-URI prefix, often requiring dynamic base URL rewriting (e.g., passing --base-url /node/$(hostname)/$(port) to the application), which severely complicates generalized deployment across different universities[1].
- **Relative Proxying (rnode URI):** A request structured as /rnode/<host>/<port>/<path> triggers the Lua scripts within mod_ood_proxy to strip the proxy identifier and host information entirely, sending only the relative path to the backend[1]. A request to /rnode/compute01/8080/api/run arrives at the backend simply as /api/run[1].

Proxy	OOD URI Structure	Path Received by	Architectural Ideal

Configuration		Backend Web Server	Use Case
Node (Absolute)	/node/compute01/8080/api/run	/node/compute01/8080/api/run	Applications explicitly configured with dynamic sub-URI base URLs.
Rnode (Relative)	/rnode/compute01/8080/api/run	/api/run	Frameworks expecting root-level relative paths (preferred for generalized APIs).

For a generalized, user-friendly deployment, utilizing the /rnode/ configuration is the optimal strategy. It allows the underlying Python FastAPI server deployed on the compute node to operate seamlessly without needing to dynamically rewrite its base URL or internal route handlers, acting exactly as if it were receiving traffic directly from the internet[1].
To ensure the payload is routed accurately, OOD administrators restrict reverse proxy targets using a host_regex configuration within the ood_portal.yml file, typically restricting proxying to internal compute node domains for security (e.g., host_regex: '[\w.-]+\.wayne\.edu')[1]. The extension's background worker dynamically interrogates the active browser tabs, identifies the open Wayne State OOD session, and extracts the dynamically assigned compute <host> and <port> from the URL string[1]. It then constructs the precise /rnode/ path, ensuring the payload routes to the exact job allocated to the user[1].

Security Considerations of OOD Proxying

While OOD proxying is highly effective, it introduces specific security parameters that the architecture must navigate. Beginning in Open OnDemand version 4.1, the platform introduced the strip_proxy_cookies and strip_proxy_headers directives within the ood_portal.yml configuration[5]. This security measure actively strips sensitive authentication cookies (such as mod_auth_openidc_session) before proxying the request to the interactive backend application[5]. While this enhances overall system security by preventing origin servers from accessing edge authentication tokens, the browser extension must ensure that any proprietary state-tracking relies on payloads embedded within the HTTP JSON body rather than custom cookies, as the OOD proxy will sanitize unexpected header data during transit[31].

Dynamic Institution Configuration and Slurm Integration

The original premise of this research assumes the existence of a functional VS Code extension blueprint—originally developed for the Wayne State University grid—that handles the intricacies of remote execution[1]. To achieve true browser-agnostic and institution-agnostic deployment, the highly specific protocols of this blueprint must be abstracted into a universally applicable format.

Adapting the Wayne State Blueprint

The foundational blueprint operates by managing SSH configuration blocks, establishing persistent connections to the Wayne State login node (warrior), submitting jobs via sbatch, and writing requests to a shared Panasas ActiveStor or CephFS filesystem[1]. It manages an Apptainer sandbox, binds host paths (like /tmp to /dev/shm), and relies on specific shared group volume paths (e.g., /rs/rs_grp_oschome/shared/ashley/)[1].

While highly effective, this protocol is deeply entangled with Wayne State specifics, such as the voh3.nas.grid.wayne.edu proxy jump and the CentOS 7 (glibc 2.17) host operating system requiring legacy compatibility libraries[1]. By replacing the SSH tunneling architecture with the OOD rnode proxy method discussed above, the network topology is immediately generalized. However, the workload management and storage architecture must still be dynamically mapped.

Autonomous Schema Mapping

To generalize this for any university cluster, the architecture dictates a dynamic translation facilitated by an LLM parsing the target university's technical documentation[1]. The user uploads their institution's specific HPC documentation (wiki pages, Slurm configuration parameters, etc.) to the local Claude instance. The LLM is instructed to map these specific idiosyncrasies against a generalized JSON template.

This process outputs a standardized **Dynamic Institution Configuration Schema**, which isolates:

1. **Workload Management:** Identifies standard CPU, GPU, and requeue/preemptible Slurm partition names, adjusting the worker.sbatch.tpl job template accordingly[1].
2. **Storage Architecture:** Maps high-speed scratch space paths (e.g., /tmp, /export/tmp, or $TMPDIR), read-only dataset mounts, and identifies the underlying file system type[1].
3. **Containerization Runtime:** Determines explicit configuration flags required for Apptainer, particularly regarding unprivileged user namespace support[1].

Interfacing with the Slurm Workload Manager

Once the payload arrives at the compute node, it must interact with the cluster's workload manager to dispatch subsequent analytical tasks. The generated schema dictates how the backend interacts with Slurm. Open OnDemand traditionally interfaces with Slurm by issuing Command Line Interface (CLI) commands, defaulting to sbatch wrappers with the --export=NONE flag, which prevents the user's host environment from polluting the job execution[34].

Alternatively, modern Slurm deployments (version 20.11 and newer) feature the slurmrestd daemon, which provides a fully featured REST API[38]. The slurmrestd daemon operates synchronously with the Slurm controller (slurmctld) and is entirely stateless, discarding all state associated with a request once the HTTP response code is sent[38]. If the generated schema detects REST API availability, the backend can abandon fragile bash-script wrappers and

instead authenticate with slurmrestd using JSON Web Tokens (JWT) to submit and monitor jobs programmatically[38]. Because JWTs function symmetrically to traditional Munge authentication based on UID/GID mappings, the familiar Slurm user/group definitions remain intact[39].

Containerization, Unprivileged Execution, and Storage Constraints

Deploying containerized LLM workloads in HPC environments requires navigating strict security policies that differ drastically from standard enterprise Docker deployments. Upon arrival at the OOD reverse proxy, the HTTP payload is routed to a lightweight Python FastAPI server running on the user's allocated compute node. This server acts as the primary orchestrator, but it must operate within a tightly controlled containerized sandbox.

Apptainer and the Fakeroot Paradigm

HPC grids strictly rely on Apptainer (formerly Singularity) due to stringent security requirements regarding root-level privileges[1]. Apptainer operates under an "integration over isolation" philosophy, meaning the user inside the container is identical to the user outside the container by default[1]. It does not manage cgroups or enter new PID, network, or IPC namespaces unnecessarily, ensuring that containerized processes communicate seamlessly with the host's batch scheduling system[1].

Running Python packages required for LLM execution often demands root-like capabilities during setup. The generalized architecture utilizes the --fakeroot flag[1]. When Apptainer is run with --fakeroot, it relies on unprivileged user namespaces. It maps UIDs and GIDs inside a user namespace to different host IDs (defined by the administrator in /etc/subuid and /etc/subgid), simulating root privileges inside the container[1]. This allows the LLM worker to utilize standard package managers (like apt or pip) directly on the compute node without possessing actual root privileges on the host kernel[1].

OverlayFS and Storage Bottlenecks

HPC workloads written by LLMs require significant writable scratch space for tensor caching and intermediate data processing. Apptainer utilizes the kernel's OverlayFS to provide a writable layer on top of a read-only SIF (Singularity Image Format) image[1].

If the host environment permits, the backend dynamically provisions a sparse EXT3 overlay image utilizing the apptainer overlay create --sparse command[1]. A sparse overlay image only takes up space on disk as data is written to it, significantly optimizing storage quotas. When the container is launched with the --overlay flag, it unlocks write capabilities across the internal system paths without modifying the immutable shared image[1].

However, the dynamic translation protocol must actively account for underlying storage limitations. Network parallel file systems common in HPC, such as Panasas ActiveStor (used at Wayne State) or Lustre, frequently do not support UID/GID mapping within user namespaces effectively[1]. Consequently, placing writable overlays or cache directories (like the SINGULARITY_CACHEDIR environment variable) on these network mounts while utilizing --fakeroot results in catastrophic "permission denied" errors because the file server is not aware of the localized namespace mappings[1]. The system must rigorously verify the storage

architecture and default to localized node storage, such as /tmp (which Wayne State maps to the transient $TMPDIR), to ensure execution stability when user namespaces are enforced[1].

Execution Mechanism	Required Capabilities	Advantages	HPC Limitations
--fakeroot Execution	Unprivileged user namespaces, UID mapping in /etc/subuid	Allows package installation, mimics root access natively	Fails on Panasas/Lustre storage; often disabled by HPC admins[1].
OverlayFS Ext3 Image	Kernel OverlayFS support, mkfs.ext3	Persistent writable layer, high metadata performance	Fixed file size; writes fail once the image is full[1].
Sandbox Directory	Standard file system permissions	Universally compatible; easy to debug without namespaces	Slow startup times; high metadata load on parallel file systems[1].

Security Landscape and Vulnerability Mitigation

Apptainer administrators are highly sensitive to vulnerabilities involving unprivileged user namespaces and OverlayFS. The architecture's reliance on --fakeroot and localized EXT3 sparse overlays perfectly aligns with modern, hardened administrative constraints designed to mitigate severe Common Vulnerabilities and Exposures (CVEs).

Recent high-profile vulnerabilities highlight the inherent risks of manipulating file permissions within namespaces. For example, CVE-2023-0386 exposed a local privilege escalation flaw in the Linux kernel's OverlayFS[49]. It allowed an attacker to create a FUSE file system containing a malicious binary owned by root with the Setuid (SUID) bit set, enter a new user namespace to mount an overlay file system, and trick the kernel into copying the malicious binary into a world-writable directory on the host[49].

Similarly, CVE-2026-48785 revealed an Apptainer path traversal vulnerability that allowed local users to bypass path restrictions specifically when Apptainer was executing in setuid mode[50]. Because of vulnerabilities like these, many modern HPC clusters—such as the LUMI supercomputer—force Apptainer into non-suid mode and entirely disable user namespaces due to security concerns regarding kernel exploitation[50].

By operating entirely within the constraints of unprivileged execution, avoiding the setuid-root component of Apptainer, and relying on the pre-authenticated secure tunnels provided by the OOD reverse proxy, this architecture ensures that the LLM execution environment remains isolated and strictly adheres to the principle of least privilege[52]. If the Dynamic Institution

Configuration Schema detects that --fakeroot is unsupported (as seen in highly restrictive environments like LUMI), the system gracefully falls back to extracting the SIF container into a standard sandbox directory, sacrificing some startup speed for universal, secure compatibility[1].

Deterministic Execution via Zod Schema Contracts

Once the prompt and associated multimodal files are successfully staged on the compute node, the system must ensure that the LLM operates deterministically. In a conversational web interface, models tend to generate extensive conversational padding. In an automated HPC environment, this padding causes catastrophic parsing failures when the system expects executable Python automation scripts or Slurm configuration files.

Treating LLM output as a strongly typed REST API response is a fundamental architectural requirement[1]. The system leverages validation libraries, specifically Zod (with Pydantic utilized concurrently on the Python backend), to enforce a rigid schema contract[1]. Zod serves as the single source of truth, deriving both the runtime validation rules and the underlying TypeScript types automatically, preventing the data contract from diverging over time[1].

To prevent the LLM from hallucinating commands or dropping required JSON keys, the architecture utilizes context injection coupled with advanced few-shot prompting[1]. When the extension intercepts the prompt, it invisibly prepends a highly structured system instruction outlining the specific capabilities of the target HPC grid (derived from the Configuration Schema). By forcing the model to calculate its analytical steps in a designated reasoning_steps array before outputting the final executable command, the final classification tokens are conditioned on structured analysis, significantly improving the reliability of the generated code[1].

Autonomous Error Correction and Retry Loops

The parsing logic implements highly defensive strategies to extract the JSON payload. LLMs consistently wrap outputs in markdown code fences; the extraction functions strip these fences using robust substring matching[1].

Crucially, the system employs the safeParse methodology rather than standard parse methods[1]. If an LLM hallucinates an enum value—for example, attempting to submit a job to a super-gpu partition that does not exist in the Wayne State configuration—standard parsing throws a fatal syntax error that crashes the application[1]. By utilizing safeParse, the system catches the structural violation gracefully[1].

When a validation failure occurs, it triggers an autonomous error correction retry loop. The precise error message generated directly by the Zod validator (e.g., "partition must be one of: cpu-standard, gpu-a100, high-mem") is dynamically formatted and fed back into the LLM as a subsequent prompt[1]. LLMs demonstrate exceptionally high rates of self-correction when provided with exact Zod validation failures, as the explicit error string provides the necessary semantic context for the model to revise its output[1]. This feedback loop operates entirely autonomously on the compute node, shielding the user operating the claude.ai frontend from underlying syntax errors and validation loops[1].

Bidirectional Streaming Integration via FastAPI and

SSE

The final architectural hurdle is communicating the results of the HPC execution back to the claude.ai session in real-time. This requires a robust, unidirectional streaming protocol capable of tunneling through the university firewall and updating the browser DOM dynamically.

Server-Sent Events over HTTP Chunking

While WebSockets are frequently used for real-time bidirectional communication, they introduce significant network overhead and require specialized proxy configurations. Specifically, WebSockets do not utilize traditional CORS preflight mechanisms and require the Apache proxy to successfully handle Upgrade and Connection headers[58]. In complex institutional environments, intermediate firewalls or poorly configured mod_proxy_wstunnel modules frequently terminate WebSocket connections or result in silent failures[59].

Instead, the architecture utilizes Server-Sent Events (SSE)[2]. SSE is a browser-native API (part of the HTML5 specification) that provides a one-way, real-time stream from the server to the client using standard HTTP/1.1 chunked transfer encoding[60]. Because SSE operates over standard ports (80/443) and utilizes standard HTTP request-response patterns without requiring protocol upgrades, it seamlessly passes through the Open OnDemand Apache reverse proxies unaffected by typical WebSocket blocking measures[61].

Feature	Server-Sent Events (SSE)	WebSockets
Directionality	Unidirectional (Server → Client)	Bidirectional (Full Duplex)
Protocol Foundation	Standard HTTP/1.1 (Chunked)	Custom ws:// or wss://
Proxy Compatibility	Excellent (Native HTTP traversing)	Often requires specific proxy header config
Data Format	Text/Event-Stream (UTF-8)	Binary or Text
Reconnection	Native browser auto-reconnect	Requires custom implementation

The Python backend utilizes FastAPI's EventSourceResponse (integrated via the sse-starlette library) to handle the SSE stream[2]. FastAPI naturally supports asynchronous generators, allowing the route to yield data continuously while preventing the application's event loop from blocking[2]. The EventSourceResponse automatically handles proper event-stream framing, sets the necessary Content-Type: text/event-stream header, and manages keep-alive pings[2].

To ensure the stream is delivered in real-time and not buffered by intermediate Nginx or Apache OOD proxies (which would defeat the purpose of streaming by holding the data until the response finishes), the backend explicitly sets the X-Accel-Buffering: no and Cache-Control: no-cache headers[6].

Mimicking the Claude.ai Response Stream

The ultimate mechanism for making this workflow clean and user-friendly lies in how the data is formatted during the SSE transmission. The official Anthropic Claude API inherently uses SSE to deliver text incrementally to the frontend application[65]. When the WXT browser extension intercepts the outbound prompt and mocks the fetch response, it must return a ReadableStream that perfectly mimics the structural shape of Anthropic's proprietary events[65].

The FastAPI server is programmed to wrap the output of the HPC workload into specific SSE event strings. Anthropic's event flow follows a strict sequence: it begins with a message_start event containing empty content, followed by a content_block_start event[66]. As the HPC script executes—for instance, tailing a Slurm log file, compiling code, or outputting data analysis—FastAPI yields the stream incrementally as content_block_delta events, specifically embedding the data inside a text_delta object[65]. Finally, it concludes the stream with a content_block_stop and a message_stop event[66].

Because the mocked HTTP response provided by the injected MAIN world script perfectly mirrors the expected JSON structure of Anthropic's backend, the React-based frontend of claude.ai naturally parses the incoming stream without throwing format errors[65]. The HPC execution logs, scientific calculations, and analytical results appear on the user's screen with the exact same typewriter effect as a standard LLM response[1]. This deep protocol emulation completely masks the complex underlying redirection to the university grid, allowing the researcher to interact with high-performance computational resources as simply as chatting with an AI assistant.

Conclusion

The architecture detailed in this report represents a highly sophisticated, rigorously secure method for abstracting university-specific HPC details to enable LLM orchestration. By utilizing the WXT framework to deploy a MAIN world DOM interception script, the system seamlessly captures text and multimodal data payloads directly from the claude.ai interface, entirely bypassing the need for complex desktop integrations.

The architecture elegantly removes traditional authentication friction by capitalizing on the browser's shared cookie state. By tunneling payloads securely through Open OnDemand's rnode reverse proxy using native credentials: 'include' fetch directives, the system establishes a secure bridge directly to an allocated compute node. Furthermore, by employing Apptainer with --fakeroot mapped to localized EXT3 sparse overlays, the system successfully circumvents strict HPC security policies and network file system constraints while mitigating exposure to kernel-level vulnerabilities like CVE-2023-0386. Finally, the integration of Zod schema validation for autonomous error correction, combined with Server-Sent Events engineered to perfectly mimic Anthropic's proprietary stream structure, results in a seamless, real-time user experience. This paradigm effectively transforms a consumer-facing web application into a powerful,

localized scientific computing interface.

Works cited

1. University-Agnostic HPC AI Architecture.pdf
2. Implementing Server-Sent Events (SSE) in FastAPI - Toby Devlin, https://tobydevlin.com/blog/sse-server-sent-events-in-fastapi/
3. Configuration — Open OnDemand 1.5.5 documentation, https://osc.github.io/ood-documentation/release-1.5/infrastructure/ood-portal-generator/configuration.html
4. CORS Cookie not set on cross domains, using fetch, set credentials, https://stackoverflow.com/questions/56965476/cors-cookie-not-set-on-cross-domains-using-fetch-set-credentials-include-an
5. ood_portal.yml — Open OnDemand 4.2.0 documentation, https://osc.github.io/ood-documentation/latest/reference/files/ood-portal-yml.html
6. Server-Sent Events (SSE) - FastAPI, https://fastapi.tiangolo.com/tutorial/server-sent-events/
7. Claude MCP Unveiled: Building a Powerful Image Analysis Tool, https://medium.com/@paulsa894/claude-mcp-unveiled-building-a-powerful-image-analysis-tool-with-anthropics-multimodal-ai-guide-f5f956158174
8. What Is a Browser Extension? Build Your First One with WXT, https://redreamality.com/blog/browser-extension-development/
9. WXT: Next-gen Web Extension Framework, https://wxt.dev/
10. Content Scripts - WXT, https://wxt.dev/guide/essentials/content-scripts
11. Web Extensions - W3C on GitHub, https://w3c.github.io/webextensions/specification/
12. scripting.ExecutionWorld - Mozilla - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/scripting/ExecutionWorld
13. Manifest - content scripts | Chrome Extensions, https://developer.chrome.com/docs/extensions/reference/manifest/content-scripts
14. Build a Next-Gen MV3 Chrome Extension with WXT and Modern, https://medium.com/insiderengineering/build-a-next-gen-mv3-chrome-extension-with-wxt-and-modern-frameworks-from-setup-to-publishing-cc9c3394a73d
15. MV3 Scripting: When executeScript Needs MAIN Context (2026), https://errorfirst.com/systems-architecture/mv3-scripting-when-executescript-needs-main-context-2026/
16. Developers Insight On Manifest v3 Privacy and Security ... - arXiv, https://arxiv.org/pdf/2507.13926
17. abhimanyu-sikarwar/claude-extension - GitHub, https://github.com/abhimanyu-sikarwar/claude-extension
18. Claude AI File Upload and Reading: formats, limits, and operational, https://www.datastudios.org/post/claude-ai-file-upload-and-reading-formats-limits-and-operational-structure
19. Vision - Claude Platform Docs,

https://platform.claude.com/docs/en/build-with-claude/vision
20. Multimodal AI Tutorial: GPT-4o Vision & Audio API, https://machinelearningplus.com/gen-ai/multimodal-ai-llm/
21. How to get images from Claude into MCP server? - Reddit, https://www.reddit.com/r/mcp/comments/1hamhob/how_to_get_images_from_claude_into_mcp_server/
22. Web fetch tool - Claude Platform Docs, https://platform.claude.com/docs/en/agents-and-tools/tool-use/web-fetch-tool
23. Content scripts | Manifest V2 - Chrome for Developers, https://developer.chrome.com/docs/extensions/mv2/content-scripts
24. Content scripts | Chrome for Developers, https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts
25. Build Modern Browser Extensions with WXT, React and TypeScript, https://dev.to/seryllns_/build-modern-browser-extensions-with-wxt-react-and-typescript-h3h
26. Client Resources - Ohio Supercomputer Center, https://www.osc.edu/book/export/html/4474
27. Vulnerability Summary for the Week of July 7, 2025 | CISA, https://www.cisa.gov/news-events/bulletins/sb25-195
28. GitHub - aws-samples/openondemand-dcv, https://github.com/aws-samples/openondemand-dcv
29. Proxy problem with marimo - Get Help - Open OnDemand, https://discourse.openondemand.org/t/proxy-problem-with-marimo/4206
30. Avoid launching a web browser - Get Help - Open OnDemand, https://discourse.openondemand.org/t/avoid-launching-a-web-browser/4060
31. v4.1 Release Notes — Open OnDemand 4.2.0 documentation, https://osc.github.io/ood-documentation/latest/release-notes/v4.1-release-notes.html
32. The ttyd not working in OOD 4.1.1 - Get Help - Open OnDemand, https://discourse.openondemand.org/t/the-ttyd-not-working-in-ood-4-1-1/4757
33. Knowledge Base - Research - Wayne State University, https://services.wayne.edu/TDClient/277/Portal/KB/Category/876/Research
34. Submitting Jobs With Slurm - Research Computing Documentation, https://docs.rc.uab.edu/cheaha/slurm/submitting_jobs/
35. HPC Grid Policies - Wayne State University, https://services.wayne.edu/TDClient/277/Portal/KB/Article/20254/HPC-Grid-Policies
36. Grid FAQs - Wayne State University, https://services.wayne.edu/TDClient/277/Portal/KB/Article/20229/Grid-FAQs
37. Slurm — Open OnDemand 4.2.0 documentation, https://osc.github.io/ood-documentation/latest/installation/resource-manager/slurm.html
38. Slurm Workload Manager - REST API Details - SchedMD, https://slurm.schedmd.com/rest.html
39. Using the Slurm REST API to integrate with distributed architectures,

https://aws.amazon.com/blogs/hpc/using-the-slurm-rest-api-to-integrate-with-distributed-architectures-on-aws/
40. A Qualitative and Quantitative Analysis of Container Engines - arXiv, https://arxiv.org/pdf/2303.04080
41. Security in Apptainer — Apptainer User Guide 1.1 documentation, https://apptainer.org/docs/user/1.1/security.html
42. Fakeroot feature — Apptainer User Guide main documentation, https://apptainer.org/docs/user/main/fakeroot.html
43. Apptainer Without Setuid - arXiv, https://arxiv.org/pdf/2208.12106
44. User Namespaces & Fakeroot - Apptainer, https://apptainer.org/docs/admin/1.0/user_namespace.html
45. Persistent Overlays — Apptainer User Guide main documentation, https://apptainer.org/docs/user/main/persistent_overlays.html
46. Installing Singularity — Singularity Admin Guide 3.7 documentation, https://apptainer.org/admin-docs/3.7/installation.html
47. SingularityCE User Guide, https://docs.sylabs.io/guides/3.11/user-guide.pdf
48. Security in Apptainer — Apptainer User Guide 1.0 documentation, https://apptainer.org/docs/user/1.0/security.html
49. The OverlayFS vulnerability CVE-2023-0386 - Datadog Security Labs, https://securitylabs.datadoghq.com/articles/overlayfs-cve-2023-0386/
50. CVE-2026-48785: Apptainer Path Traversal Vulnerability, https://www.sentinelone.com/vulnerability-database/cve-2026-48785/
51. arXiv:2407.19928v1 [cs.DC] 29 Jul 2024, https://arxiv.org/pdf/2407.19928
52. Security in Apptainer — Apptainer User Guide 1.2 documentation, https://apptainer.org/docs/user/1.2/security.html
53. Security in Apptainer — Apptainer User Guide 1.3 documentation, https://apptainer.org/docs/user/1.3/security.html
54. Stop Chasing JSON: Making LLM Outputs Type-Safe in TypeScript, https://levelup.gitconnected.com/stop-chasing-json-making-llm-outputs-type-safe-in-typescript-7e121f427bf3
55. Zod + TypeScript: Schema Validation Made Easy - Telerik.com, https://www.telerik.com/blogs/zod-typescript-schema-validation-made-easy
56. Build Production-Ready AI Features with Schema-Enforced Outputs, https://dev.to/dthompsondev/llm-structured-json-building-production-ready-ai-features-with-schema-enforced-outputs-4j2j
57. Type-Safe LLM Outputs with Zod and JSON | Pritam Ghosh, https://mepritam.dev/references/constraining-llms-structured-outputs-zod-jsonschema/
58. WebSocket CORS Errors: Why They Don't Work Like HTTP, https://websocket.org/guides/troubleshooting/cors/
59. Websocket Connections Error - Page 2 - Get Help - Open OnDemand, https://discourse.openondemand.org/t/websocket-connections-error/707?page=2
60. Real-Time Notifications in Python: Using SSE with FastAPI - Medium, https://medium.com/@inandelibas/real-time-notifications-in-python-using-sse-with-fastapi-1c8c54746eb7

61. Server-Sent Events: A Comprehensive Guide | by Mohamed-Ali, https://medium.com/@moali314/server-sent-events-a-comprehensive-guide-e4b15d147576
62. Implementing Server-Sent Events (SSE) with FastAPI: Real-Time, https://mahdijafaridev.medium.com/implementing-server-sent-events-sse-with-fastapi-real-time-updates-made-simple-6492f8bfc154
63. Server-Sent Events: streaming from server to browser - Flavio Copes, https://flaviocopes.com/server-sent-events/
64. Realtime Log Streaming with FastAPI and Server-Sent Events, https://amittallapragada.github.io/docker/fastapi/python/2020/12/23/server-side-events.html
65. Claude Messages API: Streaming - Medium, https://medium.com/@gurpartap.sandhu3/claude-messages-api-streaming-4bd16a39b085
66. Streaming messages - Claude Platform Docs, https://platform.claude.com/docs/en/build-with-claude/streaming
67. Streaming Responses | Riyaz Ali, https://riyazali.net/posts/streaming-responses/
68. Response streaming · Building with the Claude API, https://academy.claude.com/courses/building-with-the-claude-api/response-streaming
69. Streaming Claude API — Key Features Explained - APIMart, https://apimart.ai/blog/streaming-claude-api-key-features-explained